

1. Project Planning & Management

1.1 Project Proposal

Executive Summary

Bookify is a web-based hotel booking platform. The system allows users to search for available rooms, view room details, and make reservations using an integrated online payment system. The system also includes an Admin Dashboard for hotel staff to manage room types, rooms, and customer bookings.

Project Objectives

1. Develop a modern, responsive hotel booking system.
2. Enable secure online payments using Stripe integration.
3. Provide an admin dashboard for managing hotel operations.
4. Follow clean architecture principles such as N-Tier, Repository, and Unit of Work patterns.

Project Scope

In Scope:

- Room booking.
- Online payment system.
- Admin dashboard.
- User authentication and role management.

Out of Scope:

- Mobile app version.
 - External shipping or logistics integrations.
-

1.2 Project Plan & Timeline

Project Duration: 4 Weeks (Nov 1, 2025 – Dec 1, 2025)

Week 1: Architecture Setup & Room Listings

- **Milestones:**
 - Set up N-Tier architecture and database.

- Create public room listings and authentication.
- **Deliverables:**
 - Database schema.
 - Login system.
 - Room listings.

Week 2: Reservation Flow & Admin Panel

- **Milestones:**
 - Implement booking flow.
 - Implement Roles and the Admin panel.
- **Deliverables:**
 - Reservation flow.
 - Admin dashboard.

Week 3: Payment & Confirmation

- **Milestones:**
 - Integrate Stripe payments.
 - Implement booking confirmation and profile page.
- **Deliverables:**
 - Payment system.
 - Booking confirmation page.
 - Profile page.

Week 4: Testing & Final Polish

- **Milestones:**
 - UI improvements.
 - Implement logging and testing.
 - Write documentation.
- **Deliverables:**
 - Final polished and tested application.

1.3 Task Assignment & Roles

Developer	Controllers / Modules	Main Responsibilities	Integration
Ahmed (Team Leader & Architect)	CompaniesController , PaymentController , InvoiceController	- Lead overall system architecture (N-Tier). - Configure Identity & Role management. - Implement Stripe payment and invoicing. - Develop Repository & Unit of Work patterns. - Review and supervise all modules.	Collaborates with Moataz on payment integration and with Abdelrahman for linking hotels to companies.
Moataz (Hotels & Rooms Developer)	HotelsController , RoomsController	- Implement CRUD operations for hotels and rooms. - Manage room types, prices, and amenities. - Handle room availability and hotel-city relationships.	Provides hotel and room data to Shahd and Amira .
Abdelrahman (Reservation & Location Developer)	ReservationController , CountriesController , CitiesController	- Develop full booking process (Create, Edit, Cancel). - Manage countries and cities (CRUD + seed data). - Validate booking dates and hotel availability.	Shares location data with Moataz and booking info with Shahd .
Shahd (Reservation & Dashboard Developer)	ReservationController , DashboardController	- Manage booking workflow and status tracking. - Create admin dashboard with	Uses data from Ahmed (payments) and Abdelrahman (reservations).

Developer	Controllers / Modules	Main Responsibilities	Integration
		charts and reports. - Integrate bookings with payments and invoices.	
Amira (Frontend Developer – User Side)	HomeController , ReviewsController	- Build user-side views for hotel search, listings, and reviews. - Apply Bootstrap, jQuery, and Toastr for UI/UX. - Connect user reviews to hotels.	Uses Actions from Moataz and Abdelrahman .
Nahla (Frontend Developer – Admin Side)	DashboardController (Views only)	- Design and implement admin dashboard UI. - Visualize booking and revenue data (Chart.js, DataTables). - Ensure a responsive layout and clean design.	Uses processed data from Shahd .

1.4 Risk Assessment & Mitigation Plan

Risk	Probability	Impact	Mitigation Plan
Project delay	High	High	Set clear milestones, weekly meetings, and use task tracker.
Code conflicts	Medium	High	Use GitHub branches and pull requests for collaboration.
Payment integration issues	Medium	High	Test Stripe in sandbox mode before production.
Poor communication	Low	Medium	Use Discord or Slack with scheduled updates.

Risk	Probability	Impact	Mitigation Plan
Data loss	Low	High	Backup database regularly and use version control.

1.5 Key Performance Indicators (KPIs)

KPI	Target	Measurement Method
Project completion rate	100% by deadline	Compare planned vs actual delivery.
System performance	Page load < 2 seconds	Use performance testing tools.
Testing success rate	90%+ test cases passed	QA test reports.
User satisfaction	≥ 90% positive feedback	User testing and survey results.
Documentation quality	100% complete and reviewed	Final report review checklist.

2. Lecturer Review

Section	Details / Lecturer's Notes
Feedback & Evaluation	
Suggested Improvements	
Final Grading Criteria	
Overall Grade	

3. Requirements Gathering

3.1 Stakeholder Analysis

Primary Stakeholders

Hotel Customers

- **Role:** End-users who search for, book, and pay for hotel accommodations.
- **Needs:** Easy-to-use interface, transparent pricing, secure payment processing, booking confirmation, and access to booking history.
- **Expectations:** Fast and intuitive booking process, responsive design, security of personal information.

Hotel Administrators

- **Role:** Internal users responsible for managing hotel inventory and customer bookings.
- **Needs:** Centralized dashboard, tools to manage rooms and types, booking status management, access to occupancy data.
- **Expectations:** Efficient workflow tools, reliable data accuracy, role-based access.

Platform Management / Super Administrator

- **Role:** Company-level administrators who oversee the entire hotel network.
- **Needs:** Tools to register and manage hotels, assign hotel admins, system analytics and reports, user and role management.
- **Expectations:** Centralized control panel, multi-hotel management, data integrity, and scalability.

Secondary Stakeholders

IT Operations Team

- **Role:** Technical staff responsible for deployment, monitoring, and maintenance.
- **Needs:** Clear deployment documentation, health monitoring endpoints, structured logging.
- **Expectations:** Production-ready application, minimal downtime, comprehensive technical documentation.

Development Team

- **Role:** Engineers building and maintaining the system.
- **Needs:** Well-defined requirements and scope, proper version control tools, clear coding standards.
- **Expectations:** Manageable complexity, reusable components, opportunity for skill development.

3.2 User Stories & Use Cases

User Stories

As a Customer:

- US-1: Browse available rooms by date range.
- US-2: View detailed room information (photos, amenities, pricing).
- US-3: Register for an account.
- US-4: Add a room to the reservation cart.
- US-5: Securely pay for reservations.
- US-6: View past and upcoming bookings.

As a Hotel Administrator:

- US-7: Add new room types.
- US-8: Create room entries linked to room types.
- US-9: View all customer bookings in a table.
- US-10: Update room availability status.
- US-11: Access restricted by role authorization.

As Hotel Management:

- US-12: Prevent double-booking of rooms.
- US-13: Monitor system health and performance.

Use Cases

UC-1 – Sign Up

- **Actor:** User.
- **Goal:** Create a new user account.
- **Main Scenario:**
 1. User clicks "Register".
 2. User enters required data (email, password).
 3. System validates email and checks for duplicates.
 4. System stores user data.
 5. User is redirected to the login page.
- **Alternative Scenarios:** Invalid input (show error), Duplicate email (show alert).

UC-2 – Log In

- **Actor:** Registered User.
- **Goal:** Authenticate user.
- **Main Scenario:**
 1. User enters email and password.

2. User clicks "Login".
 3. System validates credentials.
 4. User is redirected to the dashboard.
- **Alternative Scenarios:** Invalid credentials (show error), Forgot password (send reset link).

UC-3 – Customer Makes a Booking

- **Actor:** Customer (Guest or Registered).
- **Main Flow:**
 1. Customer browses rooms and selects dates.
 2. System displays available rooms.
 3. Customer adds room to cart and proceeds to checkout.
 4. Provides payment information (via Stripe).
 5. System confirms booking, updates room availability, and sends confirmation.
- **Alternative Flows:** Payment Failure (retry), Room Unavailable (alert).

UC-4 – Administrator Manages Room Inventory

- **Actor:** Hotel Administrator.
- **Main Flow:**
 1. Admin logs in and navigates to "Manage Rooms".
 2. Admin adds, edits, or deletes room records.
 3. System validates changes and displays a success message.

UC-5 – System Health Check Monitoring

- **Actor:** IT Operations Team.
- **Main Flow:**
 1. Monitoring service sends GET request to /health .
 2. System checks database connection.
 3. Returns HTTP 200 OK (Healthy) or HTTP 503 (Unhealthy).

3.3 Functional Requirements

FR-1: User Management & Authentication

- FR-1.1 User Registration: Account creation, validation, email uniqueness.
- FR-1.2 Login: ASP.NET Identity authentication, account lockout, password reset.
- FR-1.3 User Profile: Profile view/update, booking history.

- FR-1.4 Role-Based Access Control: Roles (Customer, Admin); restricted admin access.

FR-2: Room & Room Type Management

- Create, update, delete room types (Admin only).
- Link rooms to types, manage status and inventory.
- Display public listings with filters and pricing.

FR-3: Reservation & Booking System

- Check availability by date range.
- Maintain reservation cart in session.
- Confirm bookings with transaction safety.
- Admins can view and manage booking statuses.
- **FR-3.5: The customer must be able to cancel a booking 48 hours before the arrival date.**

FR-4: Payment Processing

- Stripe integration for secure payments.
- Tokenized payment, PCI DSS compliance.
- Handle failures and log all transactions.

FR-5: Admin Dashboard

- Overview metrics (bookings, revenue).
- DataTables for sorting/search/export.
- CRUD operations with validation and Toaster notifications.

FR-6: Notifications & User Feedback

- Toaster JS for instant alerts (e.g., "Saved successfully").
- Client-side and server-side form validation.
- **FR-6.3: Send a confirmation email to the customer after successful payment.**

FR-7: Advanced Search & Filtering (New)

- **FR-7.1: The user must be able to search for hotels by city name or hotel name.**
- **FR-7.2: The user must be able to filter results based on (price, room type, and available amenities).**

FR-8: Rating & Review System (New)

- **FR-8.1: The customer (who has completed their stay) must be able to add a rating and review for the room.**
 - **FR-8.2: Display the average rating prominently on the hotel page.**
-

3.4 Non-Functional Requirements

NFR-1: Performance

- Page load < 2s, queries < 500ms.
- Support 100+ concurrent users.
- Scalable architecture.

NFR-2: Security

- Strong authentication & RBAC.
- HTTPS/TLS enforcement.
- No sensitive card storage; tokenized payments.
- CSRF, XSS, and SQL injection prevention.

NFR-3: Reliability & Availability

- 99.5% uptime.
- Unit of Work pattern; error logging via Serilog.
- **NFR-3.3: Ensure Data Integrity, such as preventing double booking of the same room for the same date.**

NFR-4: Usability

- Clean, consistent UI/UX with responsive Bootstrap design.
- Accessibility support.
- Localization and mobile-friendly support.
- **NFR-4.4: The application must function correctly on major browsers (Chrome, Firefox, Safari).**

NFR-5: Maintainability

- SOLID principles, clean architecture.
- 70%+ code coverage for business logic.
- Full technical documentation.

NFR-6: Monitoring & Observability

- /health endpoint.
- Structured logging (Serilog).
- Analytics for performance and conversion tracking.

NF-7: Deployment & Operations

- Multi-environment setup (Dev, Staging, Prod).
- CI/CD support.
- EF migrations and Seeding.
- Cloud or on-premises deployment readiness.
- **NFR-7.5: Must have a periodic (daily) database backup plan.**